

NATIONAL CYBER EMERGENCY RESPONSE TEAM OF PAKISTAN (PKCERT)

Government of Pakistan

MOBILE APPLICATION SECURITY ASSESSMENT REPORT

Document Version	1.0
Document Name	DIVA Android Application – Static Security Analysis Report (Week 5)
Document No	PKCERT/INTERN-STT/2026/WK5/001
Classification	CONFIDENTIAL
Issuing Authority	National CERT of Pakistan (PKCERT) — Internship Program
Date of Issuance	26 August 2026

Important Note

This report reflects the security posture of the assessed application only at the time of assessment. Any subsequent modification, update, or rebuild of the application may affect its security posture and, therefore, may render the findings and conclusions of this report invalid. This assessment was limited to static analysis; no runtime/dynamic testing is covered in this report.

This assessment was carried out as part of a supervised internship exercise on a deliberately vulnerable training application (DIVA).

Executive Summary

This report covers Week 5 of the Mobile Application Security track of the internship. The target used for this exercise was DIVA (Damn Insecure and Vulnerable App), an Android application that is built on purpose with common mobile security weaknesses, so it is safe to use for practice.

A static analysis was performed, meaning the application was examined without running it. Three tools were used together as a chain, each with a different job: MobSF was used to automatically scan the APK and find possible issues; JADX was used to read the Java-like source code and understand and manually verify what MobSF found; and Apktool was used to decode the raw APK and independently confirm the same findings at the AndroidManifest, Smali, and resource level.

The automated scan gave the application a security score of 36 out of 100. In total, 10 findings were identified and carried through the full verification chain: 3 High severity, 4 Medium severity, and 3 Low / Informational (including one possible secret that was investigated and reclassified after manual review).

The most significant confirmed finding was that the application writes the user's username and password in plain text to a file on external storage, which was traced and confirmed down to the exact line of Smali code that performs the file write. A second finding, initially flagged by MobSF as a possible hardcoded secret, was manually investigated using JADX and Apktool and found to actually be the name of a SharedPreferences key rather than a real password — this shows the value of manually verifying automated findings instead of accepting them at face value.

A full breakdown of each finding, the exact verification chain used, proof-of-concept evidence, and remediation recommendations are provided in the following sections.

1. Summary of Findings

1.1 Basic Information

Target Owner	PKCERT — Internal Training Lab
Target Custodian	Hafiz Muhammad Moawiz (Trainee, Software Security Testing)
Target Name	DIVA — Damn Insecure and Vulnerable App (DivaApplication.apk)
Target Type	Android Application (APK) — Static Analysis
Package Name	jakhar.aseem.diva
Target IP / URL	N/A — Mobile application assessment, no network endpoint in scope
Assessment Type	Mobile Application Security Assessment — Static Analysis (Automated + Manual Verification)
Assessment Date	26 August 2026
Tools Used	MobSF v4.5.2 (Docker), JADX 1.5.6, Apktool 2.7.0-dirty

1.2 Consolidated Vulnerability Summary

Severity	Count	% of Total
Critical	0	0%
High	3	30%
Medium	4	40%
Low / Informational	3	30%
TOTAL	10	100%

1.3 Consolidated Risk Overview

The overall risk rating for this application is High. The most serious confirmed issue is that user credentials (username and password) are written in plain text to a file on external storage, where any other application with storage permission can read them. This was independently confirmed at three levels: MobSF's automated Code Analysis flagged it as an Insecure Data Storage issue, JADX showed the relevant Java-like logic, and Apktool's Smali inspection showed the exact file-write instruction.

A second High finding is that the application is shipped with debugging enabled (android:debuggable="true"), which is not appropriate for a production build and can allow an attacker to attach a debugger and inspect or manipulate the app at runtime. This was confirmed both by MobSF's Code Analysis and directly inside AndroidManifest.xml via Apktool.

One MobSF finding — a possible hardcoded secret named "pkey" — was manually investigated rather than accepted as-is. Using JADX and Apktool, it was traced to strings.xml and to the relevant Activity classes, and found to be the name of a SharedPreferences key used to store the user's PIN, not an actual hardcoded password. This is recorded as a reclassified / informational finding, since accepting an automated tool's label without verification would have been inaccurate.

1.4 Application Footprint

The table below summarizes the application's key attributes, as reported by MobSF's Static Analyzer and independently confirmed through manual decompilation.

Attribute	Value
File Name	DivaApplication.apk
File Size	1.43 MB
Package Name	jakhar.aseem.diva
Main Activity	jakhar.aseem.diva.MainActivity
Target / Min / Max SDK	23 / 15 / 23
MobSF Security Score	36 / 100
Exported Activities	2 / 17
Exported Providers	1 / 1

2. Vulnerability Assessment

This assessment followed a three-stage chain for every finding, rather than treating each tool as a separate, independent activity:

- MobSF (FIND) — automated static scan of the APK to surface possible issues (permissions, hardcoded secrets, insecure storage, and other code-level findings).
- JADX (UNDERSTAND) — the Java-like decompiled source code was read to understand exactly what the flagged code is doing, and to manually verify whether the MobSF finding is a real issue.
- Apktool (VERIFY) — the raw APK was decoded to inspect AndroidManifest.xml, Smali code, and resources directly, giving an independent, tool-agnostic confirmation of the same finding.

Every finding below is presented with this same chain, showing what was found, how it was understood, and how it was independently verified.

2.0 Findings Summary

#	Finding	Severity	Status
1	Debug Mode Enabled in Production Build	High	Confirmed
2	Insecure Data Storage — Credentials Written to External Storage	High	Confirmed
3	Insecure Data Storage — PIN Stored in Plaintext SharedPreferences	High	Confirmed
4	Dangerous Permissions — External Storage Read/Write	Medium	Confirmed
5	SQL Injection Risk — Raw SQL Query in SQLite Usage	Medium	Confirmed
6	Exported Content Provider (NotesProvider)	Medium	Confirmed
7	Sensitive Data Written to Temporary File	Medium	Confirmed
8	Application Logs Information	Low	Confirmed
9	Backup Configuration Enabled (allowBackup=true)	Informational	Observation
10	Possible Hardcoded Secret "pkey"	Informational	Reclassified — Not a Secret

Detailed steps and the full verification chain for each finding are given below.

2.1 Debug Mode Enabled in Production Build

Severity: **High** Status: **Confirmed**

MobSF — Automated Finding:

MobSF's Code Analysis flagged "Debug configuration enabled. Production builds must not be debuggable" as a High severity issue (CWE-919, OWASP MASVS MSTG-RESILIENCE-2).

Apktool — Independent Verification:

Decoding the APK and reading AndroidManifest.xml directly confirmed the attribute `android:debuggable="true"` on the application tag.

Conclusion:

The finding is confirmed at both the automated and manifest level. A debuggable production build allows an attacker with physical or ADB access to attach a debugger and inspect memory, variables, or manipulate app behaviour at runtime.

Evidence: Figures 8 and 13

2.2 Insecure Data Storage — Credentials Written to External Storage

Severity: High Status: Confirmed

MobSF — Automated Finding:

MobSF's Code Analysis flagged "App can read/write to External Storage. Any App can read data written to External Storage" (OWASP M2: Insecure Data Storage, MSTG-STORAGE-2), linked to `InsecureDataStorage4Activity.java`.

JADX — Manual Understanding & Verification:

This finding was cross-checked using JADX by reviewing the general flow of how the `InsecureDataStorage` activities handle user-entered data, before independently confirming the exact write operation using Apktool's Smali output (below), since the raw Smali gave the clearest instruction-level proof.

Apktool — Independent Verification:

The class was located at `smali/jakhar/aseem/diva/InsecureDataStorage4Activity.smali`. The following key instructions were found in the file:

```
invoke-static {}, Landroid/os/Environment; ->getExternalStorageDirectory()Ljava/io/File;
```

This is equivalent to Java's `Environment.getExternalStorageDirectory()`, confirming the app accesses external storage. The app then constructs a file named `.uinfo.txt` inside that directory, builds a string in the form `username:password` from the values the user enters, and writes it using:

```
invoke-virtual {v1, v6}, Ljava/io/FileWriter; ->write(Ljava/lang/String;)V
```

Conclusion:

This is the strongest confirmed finding in this assessment. The full chain — MobSF flagging the storage risk, JADX providing context, and Apktool's Smali showing the exact `getExternalStorageDirectory()` call and `FileWriter.write()` instruction — proves that the application writes the user's username and password in plain text to a world-readable location on external storage.

Evidence: Figures 16 and 17

2.3 Insecure Data Storage — PIN Stored in Plaintext SharedPreferences

Severity: High Status: Confirmed

MobSF — Automated Finding:

This finding was not directly labelled by MobSF as a separate item — it was uncovered while manually investigating the "possible hardcoded secret" finding (see Section 2.10) using JADX.

JADX — Manual Understanding & Verification:

In `AccessControl3Activity.java`, the `addPin()` method obtains the user-entered PIN and stores it directly:

```
String pin = pinTxt.getText().toString(); spedit.putString(getString(R.string.pkey), pin);  
spedit.commit();
```

Later, `AccessControl3NotesActivity.java` retrieves this stored value and compares it against user input to decide whether to display private notes:

```
String pin = spref.getString(getString(R.string.pkey), ""); String userpin =  
pinTxt.getText().toString(); if (userpin.equals(pin)) { /* show private notes */ }
```

Apktool — Independent Verification:

Apktool's decoded resources were used to confirm that `R.string.pkey` resolves to the literal string "pkey", which is used purely as the `SharedPreferences` key name — see Section 2.10 for that verification.

Conclusion:

No encryption or hashing is applied anywhere in this flow — the user's PIN is stored and compared in plain text using Android's `SharedPreferences`, which is not an appropriate storage mechanism for authentication secrets.

Evidence: Figures 11 and 12

2.4 Dangerous Permissions — External Storage Read/Write

Severity: **Medium** Status: **Confirmed**

MobSF — Automated Finding:

MobSF's Application Permissions section listed 3 permissions: `android.permission.INTERNET` (Normal), `android.permission.READ_EXTERNAL_STORAGE` (Dangerous), and `android.permission.WRITE_EXTERNAL_STORAGE` (Dangerous).

Apktool — Independent Verification:

`AndroidManifest.xml`, viewed directly after decoding with Apktool, confirmed all three `<uses-permission>` entries exactly as reported by MobSF.

Conclusion:

A permission being present is not automatically a vulnerability on its own — the concern is what the application actually does with that access. As shown in Finding 2.2 above, this app does use these permissions to write sensitive credential data to external storage, which turns this permission into a real risk rather than a theoretical one.

Evidence: Figures 6 and 13

2.5 SQL Injection Risk — Raw SQL Query in SQLite Usage

Severity: **Medium** Status: **Confirmed**

MobSF — Automated Finding:

MobSF's Code Analysis raised a Warning that the application uses a SQLite database and executes a raw SQL query, mapped to CWE-89 (SQL Injection).

JADX — Manual Understanding & Verification:

The application's `SQLInjectionActivity.smali` (identified in the Smali directory listing) corresponds to a dedicated activity in DIVA built specifically to demonstrate this weakness, consistent with MobSF's finding.

Conclusion:

This finding is accepted as reported by MobSF; it is consistent with DIVA's known design as a training application. Any raw, unparameterized SQL query built from user input is a genuine SQL Injection risk.

Evidence: Figure 8

2.6 Exported Content Provider (NotesProvider)

Severity: Medium **Status:** Confirmed

MobSF — Automated Finding:

MobSF's dashboard reported 1 / 1 Exported Providers.

Apktool — Independent Verification:

AndroidManifest.xml confirmed the exact provider declaration:

Conclusion:

The NotesProvider handles application notes/data and is exported, meaning other applications on the device can potentially interact with it. This should be reviewed for proper permission-based access control, since an exported provider with no restriction can expose application data to any other installed app.

Evidence: Figure 14

2.7 Sensitive Data Written to Temporary File

Severity: Medium **Status:** Confirmed

MobSF — Automated Finding:

MobSF's Code Analysis raised a Warning that the app creates a temporary file, and that sensitive information should never be written into a temp file (OWASP M2: Insecure Data Storage, MSTG-STORAGE-2).

Conclusion:

This finding is accepted as reported by MobSF and is consistent with the broader insecure-storage pattern already confirmed independently in Finding 2.2 (credentials written to external storage).

Evidence: Figure 8

2.8 Application Logs Information

Severity: Low **Status:** Confirmed

MobSF — Automated Finding:

MobSF's Code Analysis raised an Info-level finding that the app logs information, and that sensitive information should never be logged.

Conclusion:

This is a lower-severity finding but is worth tracking, since verbose logging can leak sensitive runtime data (such as tokens, PINs, or credentials) into device logs, which other apps or a connected debugger may be able to read.

Evidence: Figure 8

2.9 Backup Configuration Enabled (allowBackup="true")

Severity: Informational **Status:** Observation

MobSF — Automated Finding:

This attribute was not separately scored by MobSF's Code Analysis list, but was noted while reviewing the manifest.

Apktool — Independent Verification:

AndroidManifest.xml confirmed android:allowBackup="true" on the application tag, alongside the debuggable attribute.

Conclusion:

This is recorded as a configuration observation rather than a confirmed vulnerability by itself. allowBackup="true" can allow app data to be included in device backups, which becomes a real risk only when combined with insecurely stored sensitive data — as is the case here with Findings 2.2 and 2.3.

Evidence: Figure 13

2.10 Possible Hardcoded Secret "pkey" — Investigated and Reclassified

Severity: Informational Status: Reclassified — Not a Secret

MobSF — Automated Finding:

MobSF's Possible Hardcoded Secrets section flagged: "pkey" : "notespin". This finding was not immediately accepted as a confirmed hardcoded password — it was queued for manual verification.

JADX — Manual Understanding & Verification:

A global text search for "pkey" was run in JADX. After filtering out unrelated Android support-library results, the relevant DIVA code was found in AccessControl3Activity and AccessControl3NotesActivity, both using:

```
sprop.getString(getString(R.string.pkey), "")
```

This showed that R.string.pkey is being used as a key name to look up a value in SharedPreferences — not as a password value itself. Tracing R.string.pkey into the decompiled resources (strings.xml) showed:

```
<string name="pkey">notespin</string>
```

Apktool — Independent Verification:

Apktool's decoded resources were checked independently to confirm the same value without relying on JADX:

```
grep -n "pkey" strings.xml 86: <string name="pkey">notespin</string>
```

Conclusion:

Both JADX and Apktool independently confirmed the same resource string, and the surrounding code context confirmed that "pkey" is the name of a SharedPreferences key, not an actual PIN or password value. MobSF's flag is therefore reclassified from "hardcoded secret" to informational/false positive. The real security issue this investigation uncovered is the plaintext PIN storage documented separately in Finding 2.3.

Evidence: Figures 9, 11, 12, 18 and 19

3. Proof-of-Concept Evidence

All screenshots below were captured live during Week 5 static analysis testing and correspond directly to the findings described in Section 2.

3.1 MobSF Setup and Automated Scan

Figure 1 — Pulling the MobSF Docker Image

```
(moawizz@moawizs-kali)-[~]
$ sudo docker pull opensecurity/mobile-security-framework-mobsf:latest
latest: Pulling from opensecurity/mobile-security-framework-mobsf
039e6f9f9752: Pull complete
dd6b9aa5a2f3: Pull complete
99d53fce4239: Pull complete
f581c1a79775: Pull complete
4ce184d09220: Pull complete
c71f4a70bc1f: Pull complete
9f936102a138: Pull complete
fb994e5e171b: Pull complete
59629638055e: Pull complete
1c2f487b188f: Pull complete
bc78377681a5: Pull complete
392de4e05f6a: Pull complete
a03e343b62bb: Pull complete
Digest: sha256:b92e31864e8b9c5decffe3297d9839b803c20693ab7940184dcf57fa64869cce
Status: Downloaded newer image for opensecurity/mobile-security-framework-mobsf:latest
docker.io/opensecurity/mobile-security-framework-mobsf:latest
```

The official MobSF Docker image (opensecurity/mobile-security-framework-mobsf) was pulled successfully on Kali Linux.

Figure 2 — Confirming the Downloaded Docker Image

```
(moawizz@moawizs-kali)-[~]
$ sudo docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
opensecurity/mobile-security-framework-mobsf	latest	32e09114b697	4 days ago	2.34GB
bkimminich/juice-shop	latest	9f0f27a44514	2 months ago	338MB
hello-world	latest	e2ac70e7319a	5 months ago	10.1kB

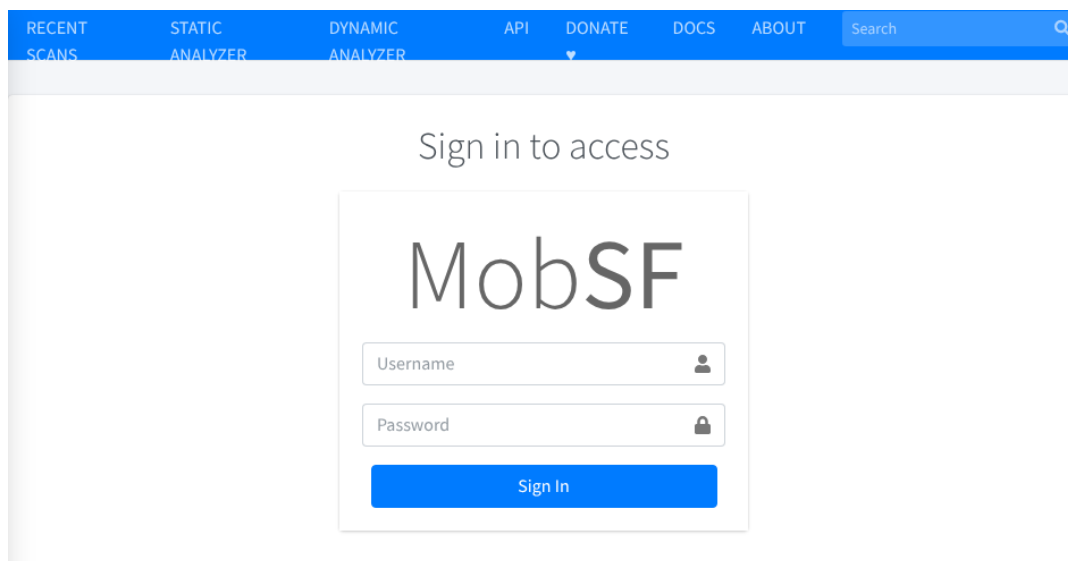
docker images confirming the MobSF image is present and ready to run.

Figure 3 — Running MobSF via Docker

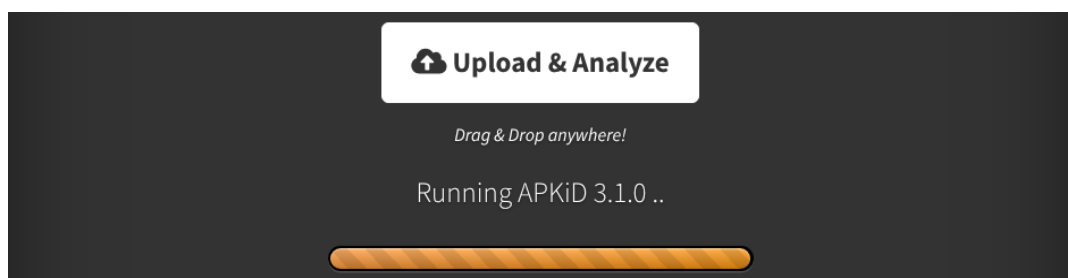
```
(moawizz@moawizs-kali)-[~]
$ sudo docker run -it --rm -p 8000:8000 opensecurity/mobile-security-framework-mobsf:latest
[INFO] 26/Aug/2026 07:53:09 - Loading User config from: /home/mobsf/.MobSF/config.py
[INFO] 26/Aug/2026 07:53:09 - Downloading JADX from https://github.com/skylot/jadx/releases/download/v1.5.0/jadx-1.5.0.zip
[##-----] 5.00%[INFO] 26/Aug/2026 07:53:12 -
```



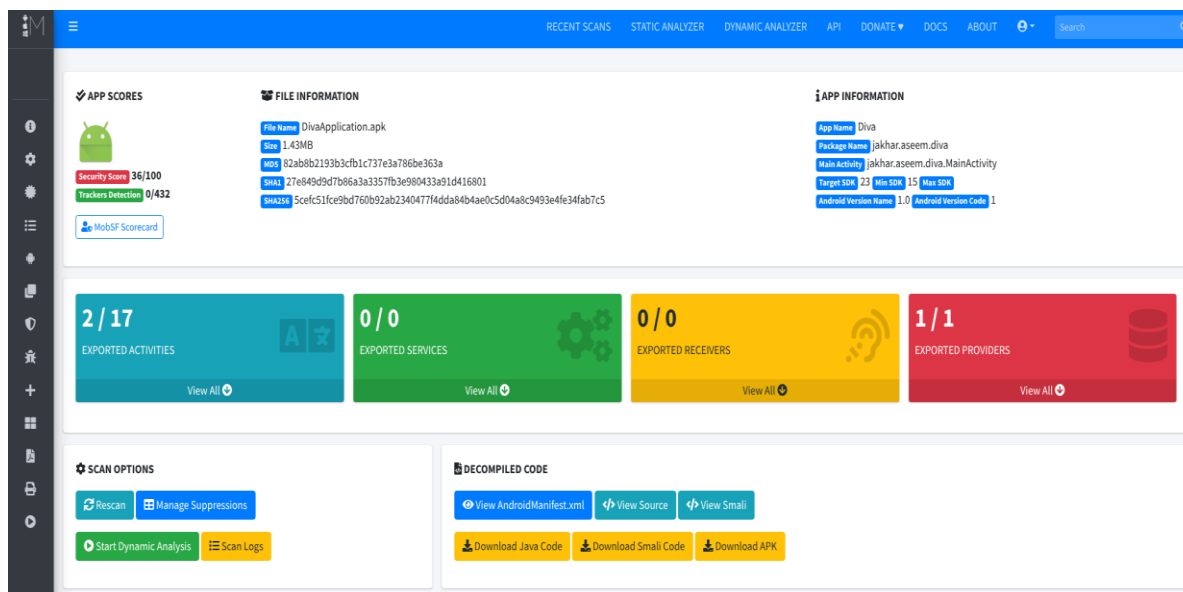
MobSF v4.5.2 started successfully on port 8000, with JADX being downloaded automatically as part of MobSF's setup.

Figure 4 — MobSF Login Page

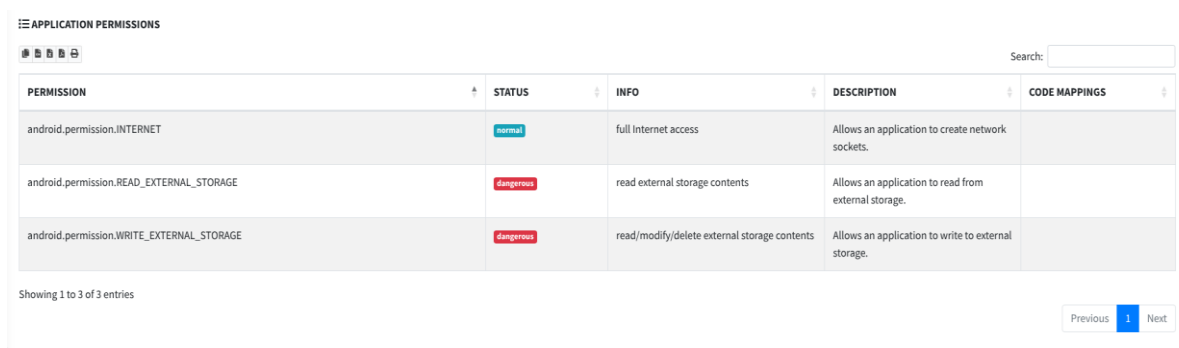
MobSF web interface accessed at localhost:8000, ready for sign-in.

Figure 5 — Uploading and Scanning the DIVA APK

DivaApplication.apk uploaded to MobSF's Static Analyzer; automated scan (APKiD) in progress.

Figure 6 — MobSF Static Analysis Dashboard

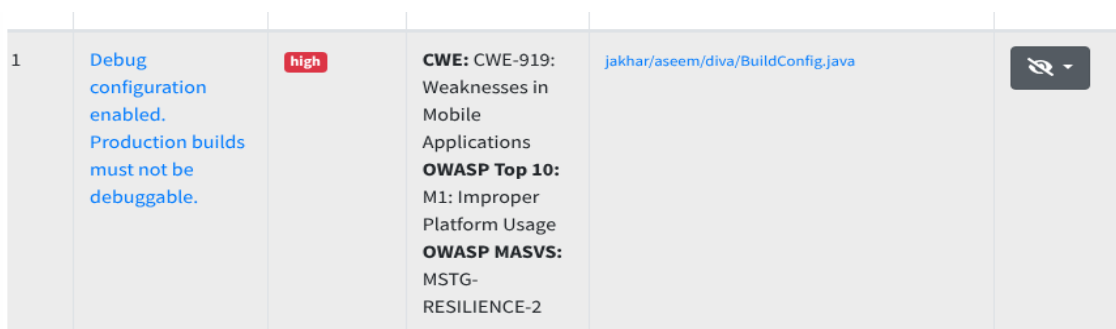
Completed scan overview: Security Score 36/100, package jakhar.aseem.diva, 2/17 Exported Activities, 1/1 Exported Providers.

Figure 7 — Application Permissions


The screenshot shows the 'APPLICATION PERMISSIONS' section of the MobSF interface. It contains a table with 5 columns: PERMISSION, STATUS, INFO, DESCRIPTION, and CODE MAPPINGS. There are 3 entries listed. Below the table, it says 'Showing 1 to 3 of 3 entries' and has 'Previous', '1', and 'Next' buttons.

PERMISSION	STATUS	INFO	DESCRIPTION	CODE MAPPINGS
android.permission.INTERNET	normal	full internet access	Allows an application to create network sockets.	
android.permission.READ_EXTERNAL_STORAGE	dangerous	read external storage contents	Allows an application to read from external storage.	
android.permission.WRITE_EXTERNAL_STORAGE	dangerous	read/modify/delete external storage contents	Allows an application to write to external storage.	

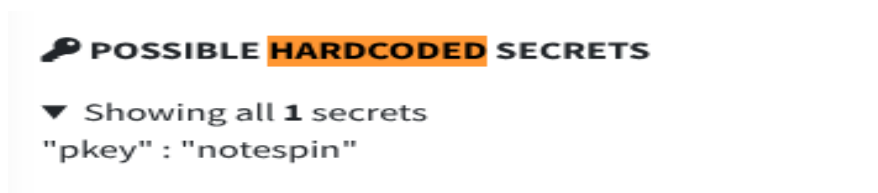
MobSF Application Permissions table listing INTERNET (Normal), READ_EXTERNAL_STORAGE and WRITE_EXTERNAL_STORAGE (both Dangerous).

Figure 8 — Code Analysis Finding: Debug Configuration Enabled


The screenshot shows a code analysis finding in MobSF. It is a table with 6 columns. The first column contains the number '1'. The second column contains the text 'Debug configuration enabled. Production builds must not be debuggable.' The third column contains a red 'high' severity tag. The fourth column contains the text 'CWE: CWE-919: Weaknesses in Mobile Applications' and 'OWASP Top 10: M1: Improper Platform Usage' and 'OWASP MASVS: MSTG-RESILIENCE-2'. The fifth column contains the file path 'jakhar/aseem/diva/BuildConfig.java'. The sixth column contains a close button.

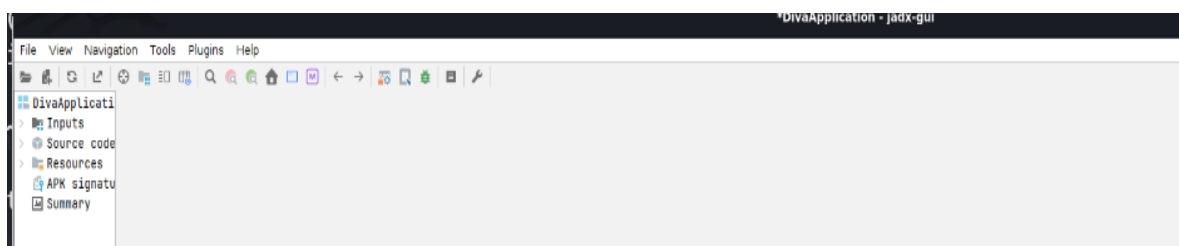
1	Debug configuration enabled. Production builds must not be debuggable.	high	CWE: CWE-919: Weaknesses in Mobile Applications OWASP Top 10: M1: Improper Platform Usage OWASP MASVS: MSTG-RESILIENCE-2	jakhar/aseem/diva/BuildConfig.java	
---	---	------	---	------------------------------------	---

MobSF Code Analysis flagging the High severity "Debug configuration enabled" issue, mapped to CWE-919 and OWASP MASVS MSTG-RESILIENCE-2.

Figure 9 — Possible Hardcoded Secrets

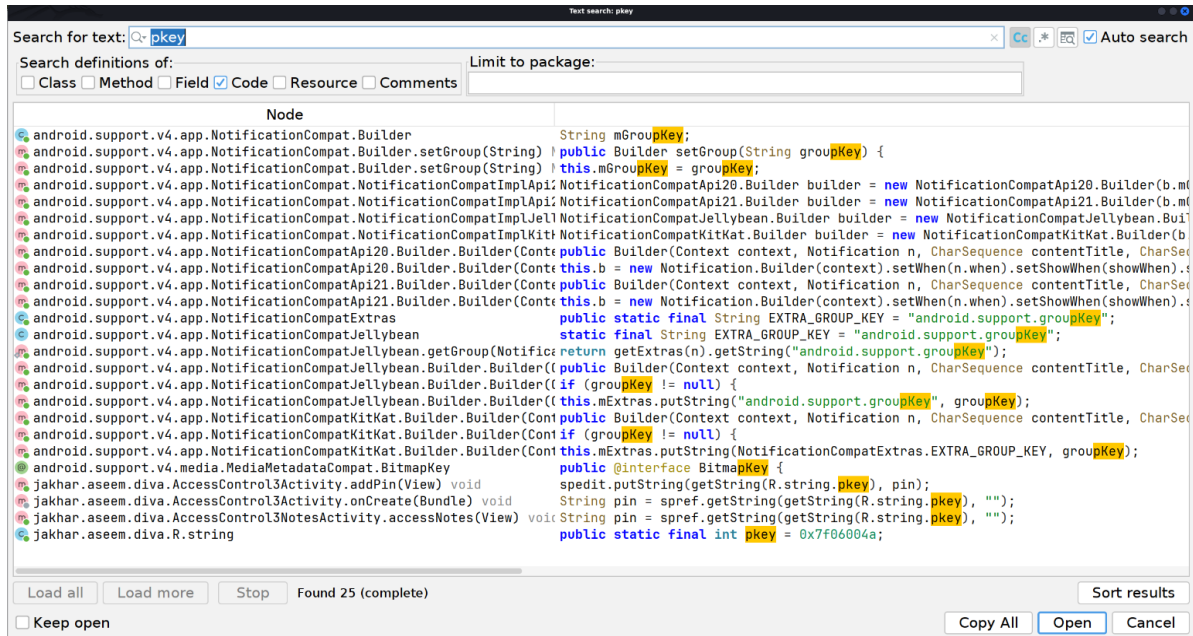
MobSF's Possible Hardcoded Secrets section flagging "pkey": "notespin" for manual investigation.

3.2 JADX — Understanding and Manual Verification

Figure 10 — JADX-GUI Loaded with the DIVA APK

DivaApplication opened in jadx-gui, showing Inputs, Source code, Resources, and Summary.

Figure 11 — Searching for "pkey" Across the Decompiled Source



JADX global text search for pkey, isolating the relevant DIVA classes (AccessControl3Activity.addPin/onCreate, AccessControl3NotesActivity.accessNotes) from unrelated Android support-library matches.

Figure 12 — Resource Verification of pkey in strings.xml

```
74 <string name="log_enter">Enter your credit card</string>
75 <string name="log_intro">Objective: Find out what the
76 <string name="notesprovider_url">content://jakob
77 <string name="pkey">notespin</string>
78 <string name="pnotes">Diva Private Notes</string>
79 </resources>
```

JADX-decompiled resources confirming `<string name="pkey">notespin</string>` — a key name, not a password value.

3.3 Apktool — Independent Verification

Figure 13 — Decoding the APK with Apktool

```
(moawizz@moawizs-kali)-[~/Downloads/DIVA]
$ apktool d DivaApplication.apk -o decoded
I: Using Apktool 2.7.0-dirty on DivaApplication.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/moawizz/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...

(moawizz@moawizs-kali)-[~/Downloads/DIVA]
$ ls
decoded  DivaApplication.apk

(moawizz@moawizs-kali)-[~/Downloads/DIVA]
$ cd decoded

(moawizz@moawizs-kali)-[~/Downloads/DIVA/decoded]
$ ls
AndroidManifest.xml  apktool.yml  lib  original  res  smali
```

apktool d DivaApplication.apk -o decoded successfully producing AndroidManifest.xml, res/, smali/, and lib/ directories.

Figure 14 — AndroidManifest.xml: Permissions, Debuggable and AllowBackup

```
(moawizz@moawizs-kali)-[~/Downloads/DIVA/decoded]
$ cat AndroidManifest.xml
<?xml version="1.0" encoding="utf-8" standalone="no"?><manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionName="6.0-2166767">
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.INTERNET"/>
    <application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name" android:theme="@style/AppTheme">
```

Manifest confirming WRITE_EXTERNAL_STORAGE, READ_EXTERNAL_STORAGE, INTERNET permissions, together with android:allowBackup="true" and android:debuggable="true".

Figure 15 — AndroidManifest.xml: Activities and Exported Provider

```
Theme">
    <activity android:label="@string/app_name" android:name="jakhar.aseem.diva.MainActivity" android:theme="@style/AppTheme.NoActionBar">
        <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category android:name="android.intent.category.LAUNCHER"/>
        </intent-filter>
    </activity>
    <activity android:label="@string/d1" android:name="jakhar.aseem.diva.LogActivity"/>
    <activity android:label="@string/d2" android:name="jakhar.aseem.diva.HardcodeActivity"/>
    <activity android:label="@string/d3" android:name="jakhar.aseem.diva.InsecureDataStorage1Activity"/>
    <activity android:label="@string/d4" android:name="jakhar.aseem.diva.InsecureDataStorage2Activity"/>
    <activity android:label="@string/d5" android:name="jakhar.aseem.diva.InsecureDataStorage3Activity"/>
    <activity android:label="@string/d6" android:name="jakhar.aseem.diva.InsecureDataStorage4Activity"/>
    <activity android:label="@string/d7" android:name="jakhar.aseem.diva.SQLInjectionActivity"/>
    <activity android:label="@string/d8" android:name="jakhar.aseem.diva.InputValidation2URISchemeActivity"/>
    <activity android:label="@string/d9" android:name="jakhar.aseem.diva.AccessControl1Activity"/>
    <activity android:label="@string/apic_label" android:name="jakhar.aseem.diva.APICredsActivity">
        <intent-filter>
            <action android:name="jakhar.aseem.diva.action.VIEW_CREDS"/>
            <category android:name="android.intent.category.DEFAULT"/>
        </intent-filter>
    </activity>
    <activity android:label="@string/d10" android:name="jakhar.aseem.diva.AccessControl2Activity"/>
    <activity android:label="@string/apic2_label" android:name="jakhar.aseem.diva.APICreds2Activity">
        <intent-filter>
            <action android:name="jakhar.aseem.diva.action.VIEW_CREDS2"/>
            <category android:name="android.intent.category.DEFAULT"/>
        </intent-filter>
    </activity>
    <provider android:authorities="jakhar.aseem.diva.provider.notesprovider" android:enabled="true" android:exported="true" and
        android:readPermission="android.permission.READ_EXTERNAL_STORAGE">
        <activity android:label="@string/d11" android:name="jakhar.aseem.diva.AccessControl3Activity"/>
```

Full activity list of DIVA together with the NotesProvider declaration showing android:exported="true".

Figure 16 — Smali Directory Listing of DIVA Activities

```
(moawizz@moawizs-kali)-[~/.../smali/jakhar/aseem/diva]
$ ls
AccessControl1Activity.smali      'NotesProvider$DBHelper.smali'
AccessControl2Activity.smali      NotesProvider.smali
AccessControl3Activity.smali      'R$anim.smali'
AccessControl3NotesActivity.smali 'R$attr.smali'
APICreds2Activity.smali           'R$bool.smali'
APICredsActivity.smali            'R$color.smali'
BuildConfig.smali                 'R$dimen.smali'
DivaJni.smali                     'R$drawable.smali'
Hardcode2Activity.smali           'R$id.smali'
HardcodeActivity.smali            'R$integer.smali'
InputValidation2URISchemeActivity.smali 'R$layout.smali'
InputValidation3Activity.smali    'R$menu.smali'
InsecureDataStorage1Activity.smali 'R$mipmap.smali'
InsecureDataStorage2Activity.smali 'R$string.smali'
InsecureDataStorage3Activity.smali 'R$styleable.smali'
InsecureDataStorage4Activity.smali 'R$style.smali'
LogActivity.smali                 R.smali
MainActivity.smali                SQLInjectionActivity.smali
```

Contents of smali/jakhar/aseem/diva/, including InsecureDataStorage4Activity.smali, used for the credential-storage verification.

Figure 17 — InsecureDataStorage4Activity.smali — External Storage Access

```

(moawizz@moawizz-kali)-[~/../smali/jakhar/aseem/diva]
$ cat InsecureDataStorage4Activity.smali
.class public Ljakhar/aseem/diva/InsecureDataStorage4Activity;
.super Landroid/support/v7/app/CompatActivity;
.source "InsecureDataStorage4Activity.java"

# direct methods
.method public constructor <init>()V
    .locals 0

    .prologue
    .line 14
    invoke-direct {p0}, Landroid/support/v7/app/CompatActivity;.<init>()V

    return-void
.end method

```

Smali source for InsecureDataStorage4Activity, showing the class definition and constructor ahead of the `getExternalStorageDirectory()` and `FileWriter.write()` calls referenced in Finding 2.2.

Figure 18 — Resources Directory Listing (Apktool)

```

moawizz@moawizz-kali:~$ cd ~/Downloads/DIVA/decoded/res
$ ls
anim  color-v11  drawable  layout-sw600dp-v13  menu  values  values-sw600dp-v13
color-v23  drawable-hdpi-v4  layout-v17  values-bn-rBD  values-km-rKH  values-sv
color-v23  drawable-ldrtl-hdpi-v4  layout-v21  values-ca  values-kn-rIN  values-sw
color-v23  drawable-ldrtl-mdpi-v4  values-en-rAU  values-cs  values-ko  values-sw600dp-v13
color-v23  drawable-ldrtl-xhdpi-v4  values-en-rGB  values-da  values-ky-rKG  values-ta-rIN
color-v23  drawable-ldrtl-xxhdpi-v4  values-en-rIN  values-de  values-large-v4  values-te-rIN
color-v23  drawable-mdpi-v4  values-es  values-el  values-ldltr-v21  values-th
color-v23  drawable-v21  values-es-rUS  values-en-rEE  values-ldrtl-v23  values-tl
color-v23  drawable-v23  values-et-rEE  values-en-rES  values-lo-rLA  values-tr
color-v23  drawable-xhdpi-v4  values-fa  values-es-rUS  values-lt  values-uk
color-v23  drawable-xxhdpi-v4  values-fr  values-et-rEE  values-lv  values-ur-rPK
color-v23  drawable-xxxhdpi-v4  values-fr-rCA  values-eu-rES  values-mk-rMK  values-uz-rUZ
color-v23  layout  values-gl-rES  values-fa  values-mn-rMN  values-v11
color-v23  layout-sw600dp-v13  values-gu-rIN  values-fr  values-ms-rMY  values-v12
color-v23  layout-v17  values-h320dp-v13  values-fr-rCA  values-my-rMM  values-v14
color-v23  layout-v21  values-h720dp-v13  values-gu-rIN  values-ne-rNP  values-v17
color-v23  menu  values-hdpi-v4  values-hi  values-nb  values-v18
color-v23  bitmap-hdpi-v4  values-hr  values-nl  values-pa-rIN  values-v21
color-v23  bitmap-mdpi-v4  values-hu  values-port  values-pl  values-v22
color-v23  bitmap-xhdpi-v4  values-hy-rAM  values-pt  values-pt-rBR  values-v23
color-v23  bitmap-xxxhdpi-v4  values-in  values-pt-rPT  values-ro  values-vi
color-v23  values  values-is-rIS  values-ru  values-si-rLK  values-w360dp-v13
color-v23  values-af  values-it  values-sk  values-sl  values-w480dp-v13
color-v23  values-am  values-iw  values-sq-rAL  values-zh-rCN  values-w500dp-v13
color-v23  values-ar  values-ja  values-sr  values-zh-rHK  values-w600dp-v13
color-v23  values-az-rAZ  values-ka-rGE  values-zh-rTW  values-w720dp-v13
color-v23  values-bg  values-kk-rKZ  values-zu  values-w820dp-v13
color-v23  values-bl  values-kn-rKH  values-zu  values-xlarge-land-v4
color-v23  values-br  values-ko  values-zu  values-xlarge-v4
color-v23  values-ca  values-ky-rKG  values-zu  values-xxlarge-v4
color-v23  values-cs  values-large-v4  values-zu  values-xxlarge-v4
color-v23  values-da  values-ldltr-v21  values-zu  values-xxlarge-v4
color-v23  values-de  values-ldrtl-v23  values-zu  values-xxlarge-v4
color-v23  values-el  values-lo-rLA  values-zu  values-xxlarge-v4
color-v23  values-en-rAU  values-lt  values-zu  values-xxlarge-v4
color-v23  values-en-rGB  values-lv  values-zu  values-xxlarge-v4
color-v23  values-en-rIN  values-mk-rMK  values-zu  values-xxlarge-v4
color-v23  values-es  values-ms-rMY  values-zu  values-xxlarge-v4
color-v23  values-es-rUS  values-my-rMM  values-zu  values-xxlarge-v4
color-v23  values-et-rEE  values-ne-rNP  values-zu  values-xxlarge-v4
color-v23  values-et-rES  values-nl  values-zu  values-xxlarge-v4
color-v23  values-fa  values-pa-rIN  values-zu  values-xxlarge-v4
color-v23  values-fr  values-pl  values-zu  values-xxlarge-v4
color-v23  values-fr-rCA  values-pt  values-zu  values-xxlarge-v4
color-v23  values-gl-rES  values-pt-rBR  values-zu  values-xxlarge-v4
color-v23  values-gu-rIN  values-pt-rPT  values-zu  values-xxlarge-v4
color-v23  values-h320dp-v13  values-ro  values-zu  values-xxlarge-v4
color-v23  values-h720dp-v13  values-ru  values-zu  values-xxlarge-v4
color-v23  values-hdpi-v4  values-si-rLK  values-zu  values-xxlarge-v4
color-v23  values-hi  values-sk  values-zu  values-xxlarge-v4
color-v23  values-hr  values-sq-rAL  values-zu  values-xxlarge-v4
color-v23  values-hu  values-sr  values-zu  values-xxlarge-v4
color-v23  values-hy-rAM  values-zh-rCN  values-zu  values-xxlarge-v4
color-v23  values-in  values-zh-rHK  values-zu  values-xxlarge-v4
color-v23  values-is-rIS  values-zh-rTW  values-zu  values-xxlarge-v4
color-v23  values-it  values-zu  values-zu  values-xxlarge-v4
color-v23  values-iw  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ja  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ka-rGE  values-zu  values-zu  values-xxlarge-v4
color-v23  values-kk-rKZ  values-zu  values-zu  values-xxlarge-v4
color-v23  values-kn-rKH  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ko  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ky-rKG  values-zu  values-zu  values-xxlarge-v4
color-v23  values-large-v4  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ldltr-v21  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ldrtl-v23  values-zu  values-zu  values-xxlarge-v4
color-v23  values-lo-rLA  values-zu  values-zu  values-xxlarge-v4
color-v23  values-lt  values-zu  values-zu  values-xxlarge-v4
color-v23  values-lv  values-zu  values-zu  values-xxlarge-v4
color-v23  values-mk-rMK  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ms-rMY  values-zu  values-zu  values-xxlarge-v4
color-v23  values-my-rMM  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ne-rNP  values-zu  values-zu  values-xxlarge-v4
color-v23  values-nl  values-zu  values-zu  values-xxlarge-v4
color-v23  values-pa-rIN  values-zu  values-zu  values-xxlarge-v4
color-v23  values-pl  values-zu  values-zu  values-xxlarge-v4
color-v23  values-pt  values-zu  values-zu  values-xxlarge-v4
color-v23  values-pt-rBR  values-zu  values-zu  values-xxlarge-v4
color-v23  values-pt-rPT  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ro  values-zu  values-zu  values-xxlarge-v4
color-v23  values-ru  values-zu  values-zu  values-xxlarge-v4
color-v23  values-si-rLK  values-zu  values-zu  values-xxlarge-v4
color-v23  values-sk  values-zu  values-zu  values-xxlarge-v4
color-v23  values-sq-rAL  values-zu  values-zu  values-xxlarge-v4
color-v23  values-sr  values-zu  values-zu  values-xxlarge-v4
color-v23  values-zh-rCN  values-zu  values-zu  values-xxlarge-v4
color-v23  values-zh-rHK  values-zu  values-zu  values-xxlarge-v4
color-v23  values-zh-rTW  values-zu  values-zu  values-xxlarge-v4
color-v23  values-zu  values-zu  values-zu  values-xxlarge-v4

```

decoded/res/ directory listing confirming the full set of value/resource folders extracted by Apktool.

Figure 19 — Grep Confirmation of pkey in strings.xml (Apktool)

```

(moawizz@moawizz-kali)-[~/../DIVA/decoded/res/values]
$ grep -n "pkey" strings.xml
86: <string name="pkey">notespin</string>

```

`grep -n "pkey" strings.xml` independently confirming the same value found via JADX, verifying the finding at the raw resource file level.

5. Recommendations

Debug Mode / Build Configuration

- Set `android:debuggable="false"` (or remove the attribute) in all production release builds.
- Ensure the CI/CD release pipeline strips debug flags automatically before signing and publishing.

Insecure Local Data Storage (Credentials & PIN)

- Never write usernames, passwords, or PINs to external storage in plain text.
- If local storage of sensitive data is required, use the Android Keystore system and encrypted storage APIs (e.g. `EncryptedSharedPreferences`).
- Do not use plain `SharedPreferences` to store PINs or any authentication secret; hash or encrypt values before storing.
- Avoid writing sensitive data to temporary files; if unavoidable, ensure the file is stored in app-private storage and deleted immediately after use.

Permissions

- Request `READ_EXTERNAL_STORAGE` / `WRITE_EXTERNAL_STORAGE` only if strictly necessary for the app's functionality.
- Where possible, use scoped storage / app-specific directories instead of shared external storage.

SQL Injection

- Replace raw, concatenated SQL queries with parameterized queries or prepared statements.
- Validate and sanitize all user input before it reaches any database query.

Exported Content Provider

- Set `android:exported="false"` for `NotesProvider` unless it must genuinely be accessible to other applications.
- If external access is required, enforce permission-based access control (`android:permission` / `android:readPermission` / `android:writePermission`) on the provider.

Logging

- Remove or disable verbose/debug logging in release builds.
- Never log credentials, PINs, tokens, or other sensitive runtime data.

Backup Configuration

- Set `android:allowBackup="false"` for applications that handle sensitive user data, to prevent that data being extracted through device backups.

6. Conclusion

Week 5 static analysis of the DIVA Android application identified 10 findings using a chained MobSF → JADX → Apktool methodology: 3 High, 4 Medium, and 3 Low/Informational. The two most significant confirmed issues are that the application writes user credentials in plain text to external storage, and that it stores a user PIN in plaintext `SharedPreferences` with no hashing or encryption — both were traced down to exact source and Smali-level evidence rather than accepted purely from automated tool output.